Codex Loop Kit

The Field Guide — claudex from end to end

From duo to autopilot.

by Mark Kashef
Early AI Dopters

What's inside

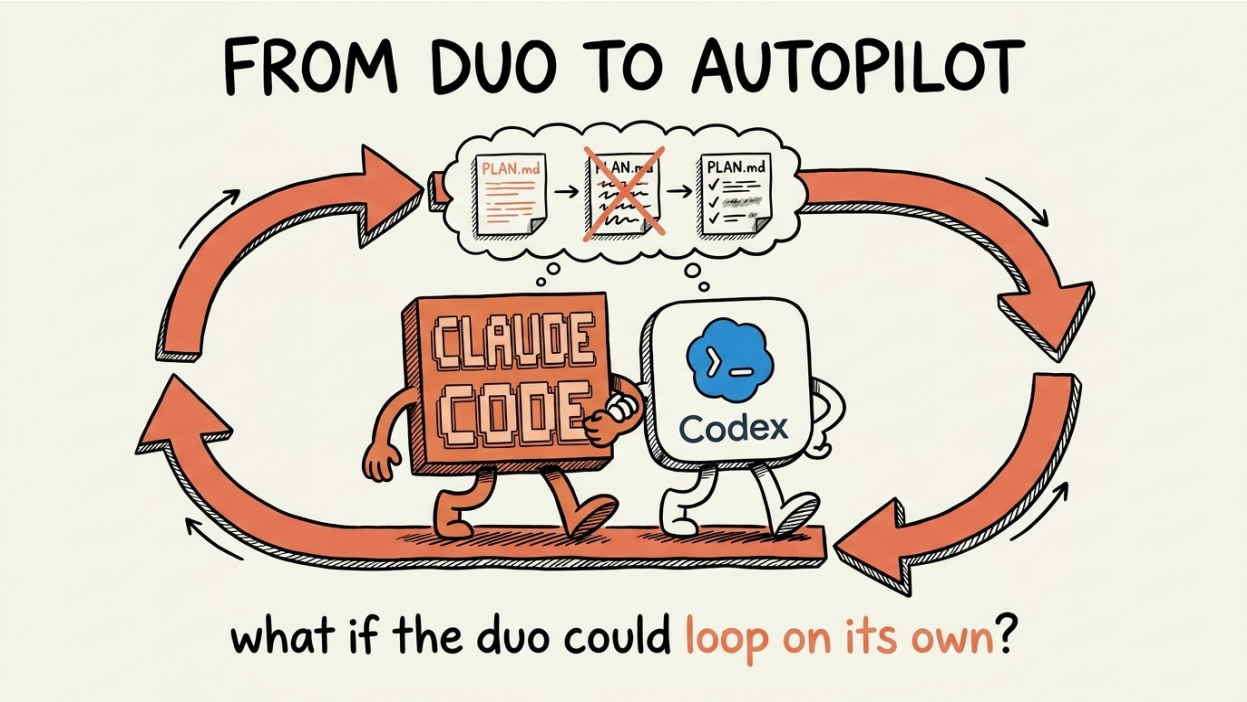
The Codex Loop video as a take-home reference. Every diagram embedded. The 50K view of how claudex loops, the X-ray of the engine beneath every slash command, all four commands at a glance, deep dives on each one, and the two real-world use cases. The companion document (Concepts Companion) goes deeper into the Stop hook architecture, state machine internals, and configuration. This guide is the lock-in-the-mental-model document. Read it once, then install claudex from github.com/promptadvisers/claudex.

Contents

01

From Duo to Autopilot

The frame for the whole video.



The previous video paired Claude Code with Codex. They make a great duo. But the duo only fires when you type a command. You're still the one driving every move, every iteration, every back-and-forth.

Claudex changes that. It wraps the duo in a loop. You type one command, hand it a topic, and the loop runs on its own. Claude drafts, Codex grills, Claude revises, on repeat until the work is clean.

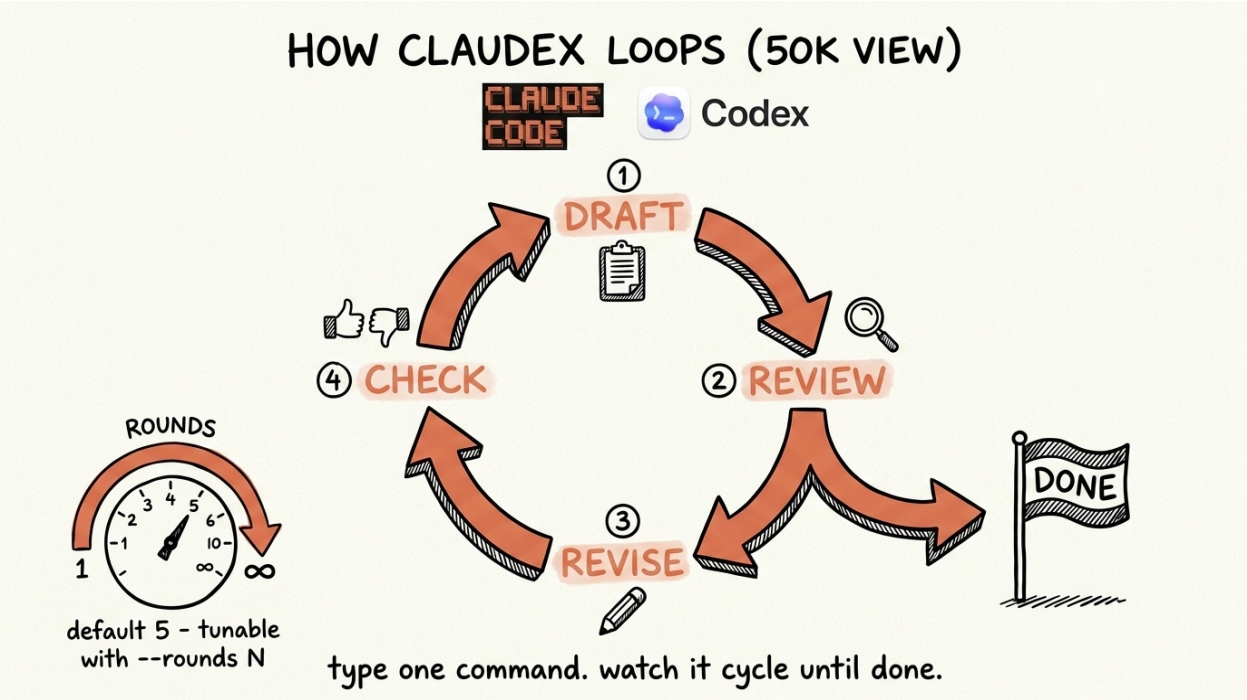
The frame

This is the autopilot the duo was missing. Same two AIs, same strengths, same tokenomics. Now you don't have to keep them moving.

02

The 50K View

How claudex loops + the tunable round count.



At 50,000 feet, claudex is a four-station loop with a dial.

The four stations:

- **DRAFT** — Claude writes a plan or a piece of code.
- **REVIEW** — Codex reads the work and produces findings.
- **REVISE** — Claude reads the findings and decides what to fix.
- **CHECK** — Did the loop reach a stopping point? If not, back to REVIEW.

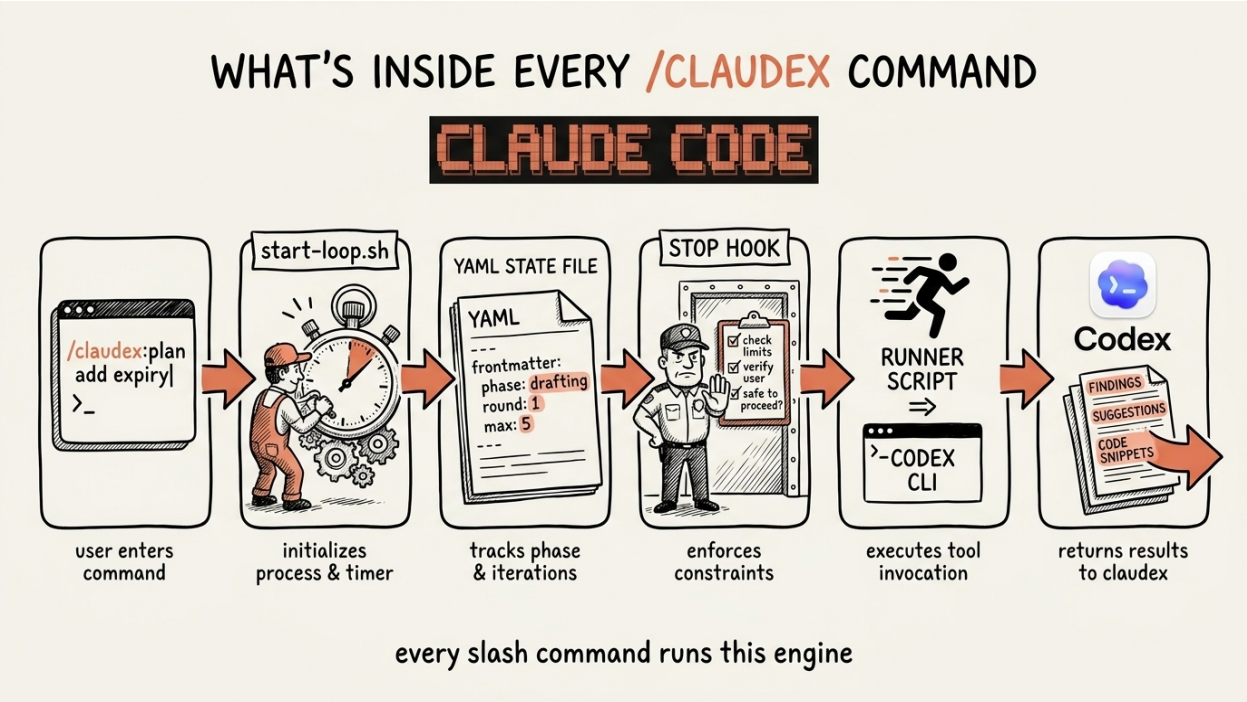
The loop continues until Codex says no substantive findings (the work is clean) OR until the round budget runs out. The default budget is 3 rounds — field-tested on real projects, three rounds catches the bulk of the issues without burning a ton of tokens.

> **The dial**
>
> Pass **--rounds 2** for a tighter loop. Pass **--rounds 5** or **--rounds 7** for a deeper one on high-stakes changes. Most people leave it at 3. The default is the right answer for nearly every case.

03

X-Ray of the Engine

What's beneath every /claudex command.



Every /claudex command is a thin wrapper around six pieces working together:

- **The slash command file** (Markdown with frontmatter) — what Claude Code reads when you type /claudex:plan.
- **start-loop.sh** — sets up state. Writes a tiny YAML file describing the loop.
- **The state file** — source of truth. Records exactly which phase the loop is in.
- **The Stop hook** — the magic. Fires every time Claude tries to finish a turn. Reads state, decides ALLOW or BLOCK.
- **Runner script** — written by the hook when it BLOCKs. Invokes Codex CLI directly with the right prompt.
- **Codex** — returns findings. Claude reads, decides, tries to finish. Hook fires again. Cycle repeats.

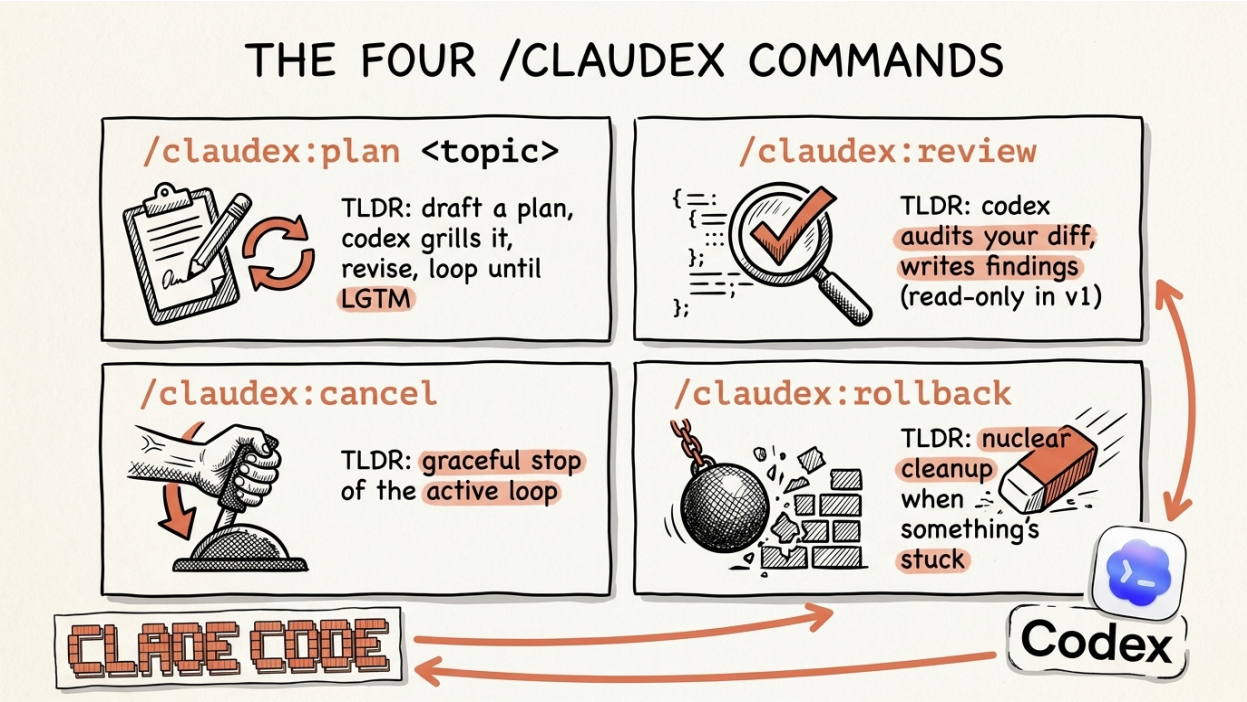
Why this works

There's no AI orchestration framework. No background agent server. No magic. Just bash, a tiny state file, and Claude Code's built-in Stop hook mechanism. The Stop hook is the only thing in Claude Code that can force a loop to keep running by BLOCKing Claude's exit. Everything else is plumbing around that one primitive.

04

All Four /claudex Commands

Memorize this slide.



The plugin gives you four commands. Two are workhorses. Two are escape hatches.

Workhorses:

- **/claudex:plan <topic>** — autonomous plan-and-review loop. Default 3 rounds, configurable with --rounds N. Optional --from-draft to use an existing PLAN.md.
- **/claudex:review** — Codex audits your current diff. Findings + proposed fixes written to reviews/. Read-only in v1.

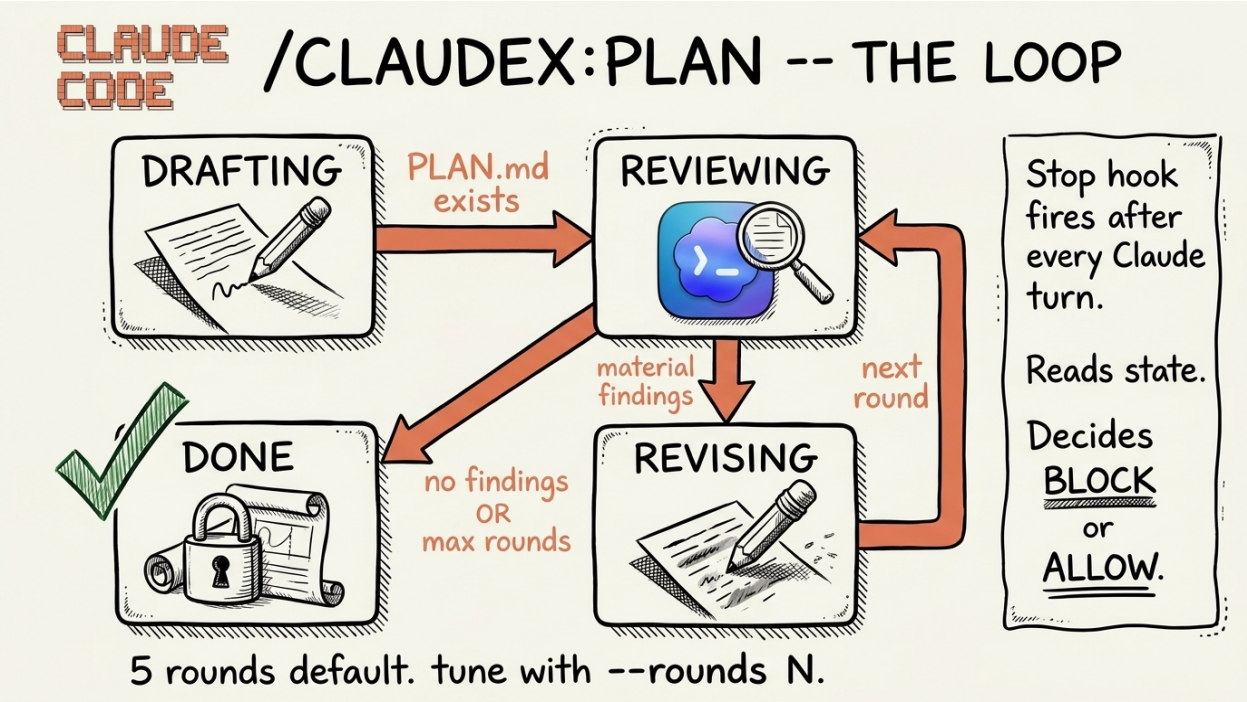
Escape hatches:

- **/claudex:cancel** — graceful stop of an active loop. Marks state cancelled. Hook allows exit on next fire.
- **/claudex:rollback** — nuclear cleanup. Wipes all state files. Use this when cancel didn't work.

05

/claudex:plan Deep Dive

The state machine + CAS transitions.



Plan mode is the most-used command. Four phases: drafting, reviewing, revising, done. Transitions between phases use compare-and-swap (CAS), which means a transition only succeeds if the current phase matches the expected phase. This prevents race conditions.

The full lifecycle:

- Round 1, drafting → Claude writes PLAN.md, ends turn.
- Stop hook fires. Sees drafting + PLAN.md exists. Transitions to reviewing. Writes runner script. BLOCK with instructions.
- Claude runs the script. Codex returns findings.
- Claude decides: revise PLAN.md (material findings) OR call mark-done.sh (no findings). Ends turn.
- Stop hook fires. If phase=done: ALLOW exit. Else: increment round, write fresh runner, BLOCK again.
- Loop continues until done OR round count exceeds max.

Sample command

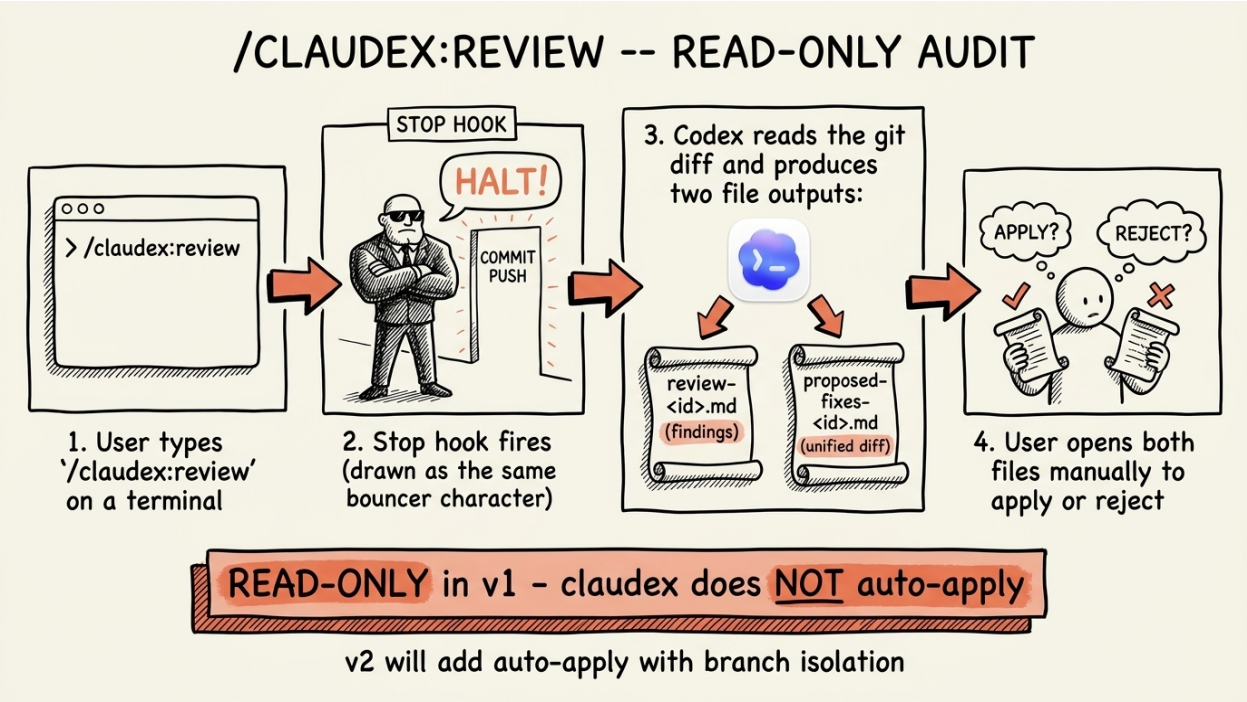
Run this in any Claude Code session with claudex installed:

```
/claudex:plan add expiry dates to my links so they auto-die
after a configured time
```

06

/claudex:review Deep Dive

Single-shot read-only audit.



Review mode is simpler than plan mode. There's no loop. The hook fires once, runs Codex against your current diff, writes two files to the reviews/ folder, and allows exit.

The two files:

- **reviews/review-<id>.md** — Codex's raw findings.
- **reviews/proposed-fixes-<id>.md** — Claude's interpretation, formatted as a unified diff.

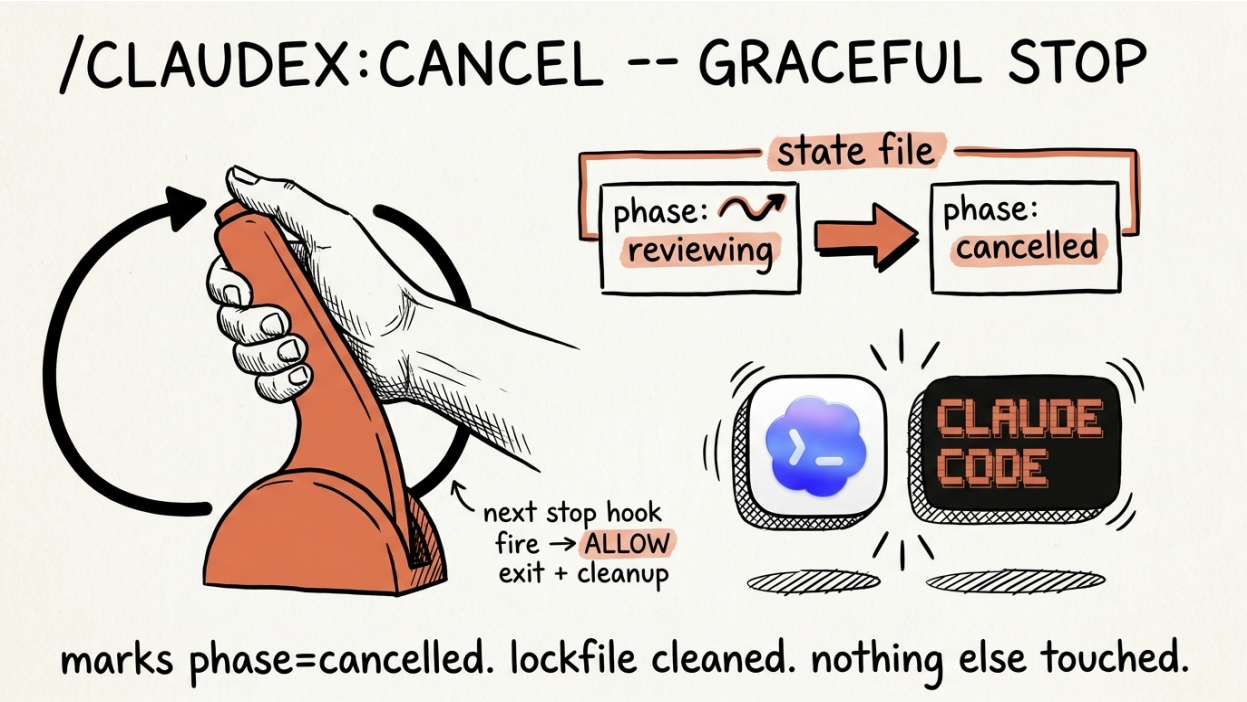
You read both, decide which patches to apply, and apply them manually with `git apply` or by hand.

> **Why read-only in v1**
>
> Auto-applying AI-generated patches into an unisolated working tree was the highest-severity finding in Codex's own review of claudex. v1 doesn't go there. v2 will, but only with explicit branch isolation. The design is in `docs/V2_DESIGN.md`.

07

/claudex:cancel

Graceful stop.



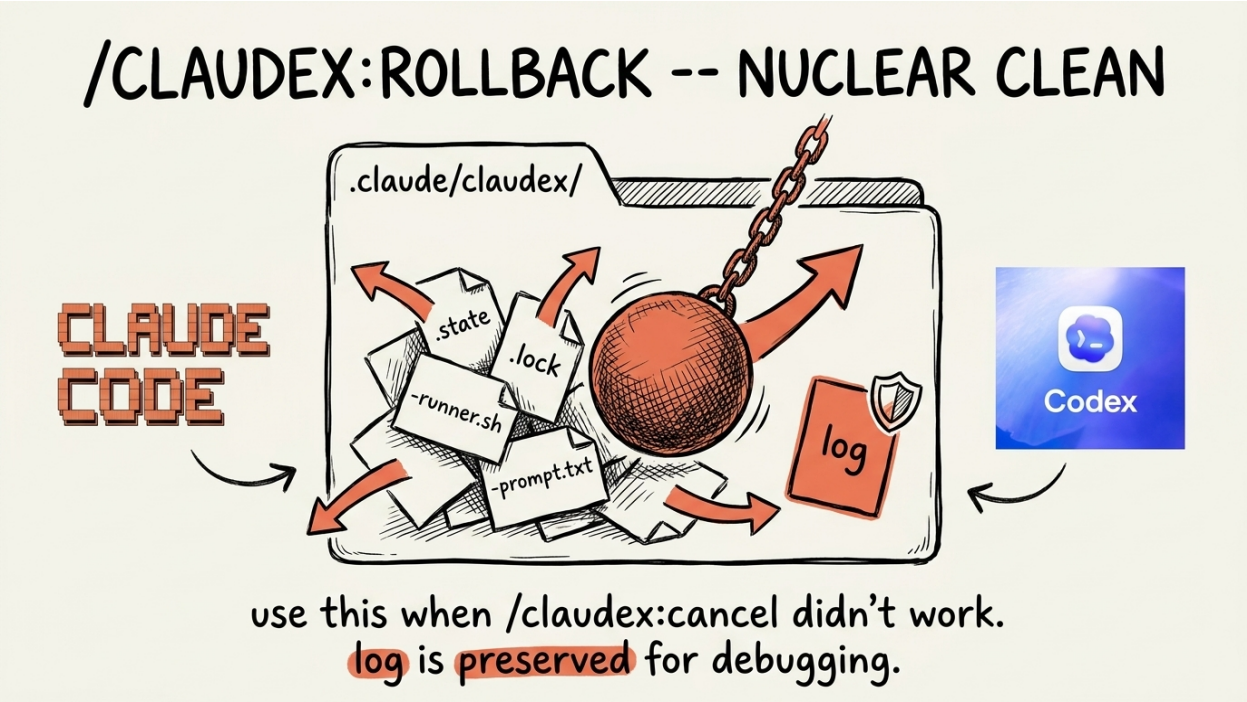
Run /claudex:cancel when you want to stop the active loop without breaking anything. The command marks the current state file's phase as cancelled and removes the lockfile.

On the next Stop hook fire, the hook sees phase=cancelled and ALLOWs exit cleanly. The state file stays on disk for audit (you can see what the cancelled loop was working on).

08

/claudex:rollback

Nuclear cleanup.



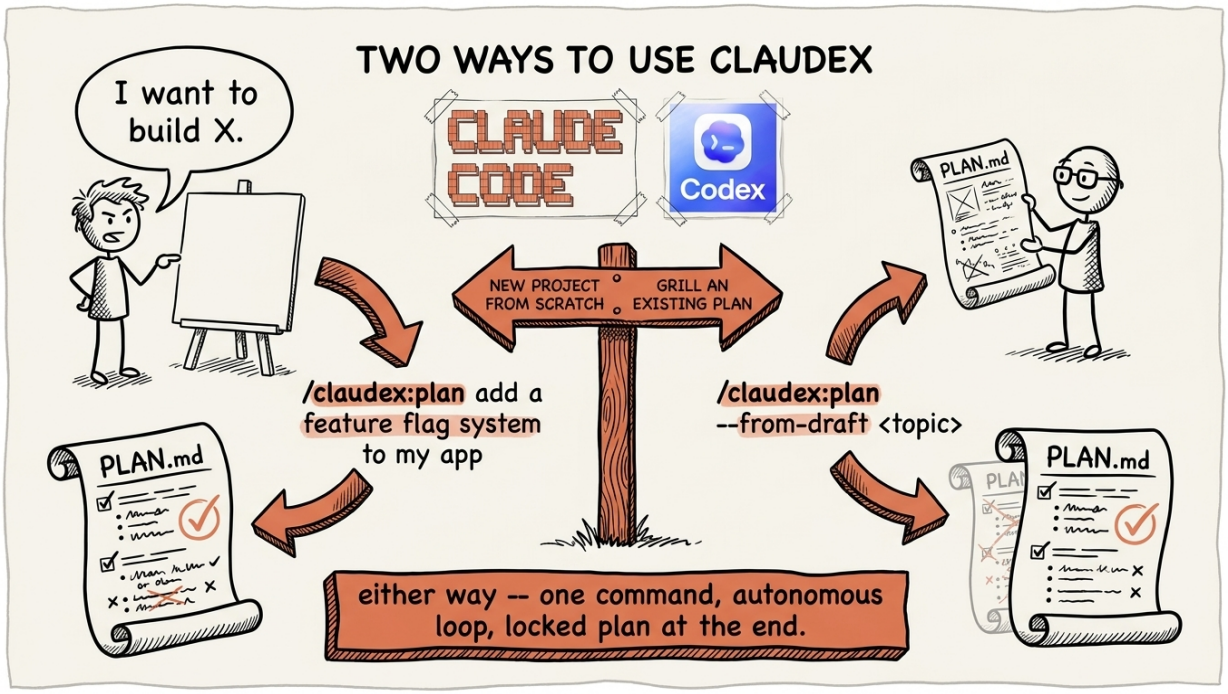
/claudex:rollback is the nuclear option. Use it when /claudex:cancel didn't work, when claudex is refusing to start a new loop because it thinks one is already active and you can't find it, or when you just want to wipe the slate.

It removes every state file, every lockfile, every runner script, every prompt artifact. The log file is preserved so you can debug what happened. Your code is untouched.

09

Two Ways to Use Claudex

The binary use case.



Path 1 — New project from scratch:

You have an idea, you want to design it well. You run /claudex:plan with a topic. Claude drafts a plan. Codex grills it. The loop iterates. You walk away with a vetted plan and a changelog showing what shifted across rounds.

```
/claudex:plan add a feature flag system to my app
```

Path 2 — Grill an existing plan:

You already wrote a draft (or someone else did, or you copied it from a doc). You drop it as PLAN.md and run /claudex:plan with --from-draft. Same loop, but it starts from your draft instead of from scratch. Codex's first round of findings will be focused on what's already there.

```
/claudex:plan --from-draft add a feature flag system to my app
```

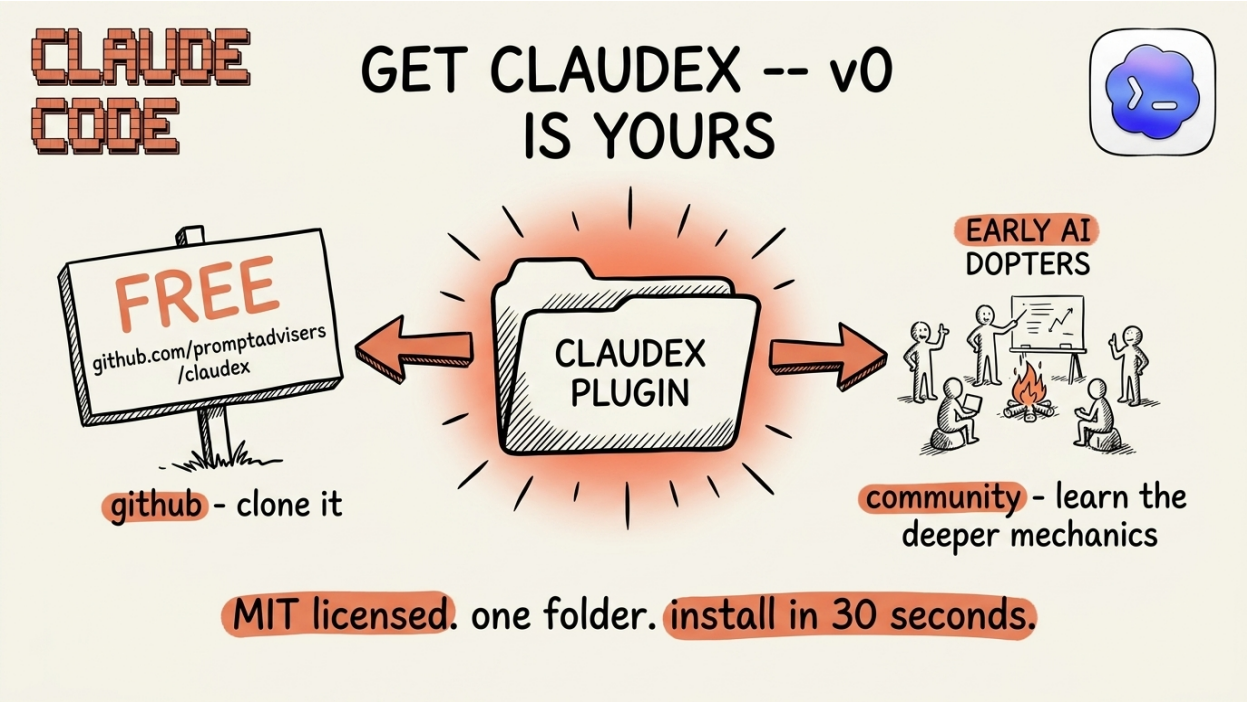
Either way

One command, autonomous loop, locked plan at the end. The work is the same; the starting point is the only thing that changes.

10

Get Claudex

Free repo + community CTA.



github.com/promptadvisers/claudex — MIT licensed. One folder. v0 is yours.

Install in 30 seconds:

```
/plugin marketplace add promptadvisers/claudex
```

```
/plugin install claudex@promptadvisers-claudex
```

```
/reload-plugins
```

If `codex` CLI isn't installed yet:

```
npm install -g @openai/codex
```

```
codex login
```

Want to extend it?

Multi-agent reviewers, custom prompts, your own slash commands, CI integration — these all live deeper than v0. The community is where we go through them together.

Join Early AI Dopters: `https://www.skool.com/earlyaidopters/about`

Cheat Sheet

Plan a feature with adversarial pushback
Default 3 rounds. Loop until LGTM.

```
/claudex:plan
```

Plan with custom round count
Tighter or deeper loop.

```
/claudex:plan --rounds N
```

Plan from existing draft
Skip the drafting step, jump straight to review rounds.

```
/claudex:plan --from-draft
```

Audit your diff
One-shot review. Findings + proposed fixes written to reviews/.

```
/claudex:review
```

Cancel an active loop
Graceful stop. State preserved for audit.

```
/claudex:cancel
```

Force-clean all state
Nuclear option. Use when cancel didn't work.

```
/claudex:rollback
```

Install the plugin
One-liner via the Claude Code plugin marketplace.

```
/plugin marketplace add promptadvisers/claudex
```

Activate after install

Then /reload-plugins.

```
/plugin install claudex@promptadvisers-claudex
```

Want the deeper teaching? Read the Concepts Companion that came in this kit. Want to extend claudex past v0? Join Early AI Dopters.

Early AI Dopters